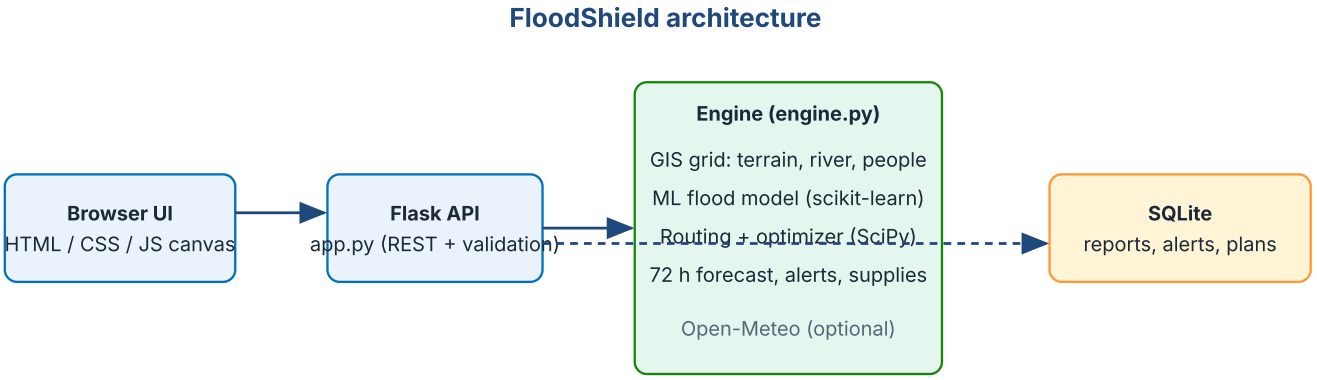
FloodShield: Technical Design Document

1. Purpose

FloodShield is a decision-support system that turns a rainfall scenario or forecast into (a) a flood-probability map, (b) a count of people at risk, (c) a shelter plan that people can actually reach, (d) a village-level early-warning timeline with multilingual alerts, and (e) a relief-supply estimate. Community reports feed back into priorities.

2. Architecture



JSON over HTTP between UI and API; engine runs in-process; SQLite stores community reports, alert log and saved shelter plans.

A static version of this diagram is in `docs/architecture.svg`.

3. Data flow (one request)

1. UI sends `POST /api/analyze {rain, shelters, capacity}`.
2. Engine predicts flood probability `P` for all 48 × 48 cells (250 m each) with the trained model.
3. A directed graph is built: edge cost = walking time x (1 + 3 x P of the cell entered); river cells are blocked except bridges; cells with P > 0.92 are impassable.
4. Multi-source Dijkstra (SciPy) from usable shelters (P < 0.5) gives walking time and route to the nearest shelter.
5. Engine returns KPIs (people affected, share reaching a shelter within 30 min, average evacuation time, capacity use), village priority table, supply estimate and route arrays. UI renders the map and panels.

4. Components

Component	File	Notes
District (GIS layers)	engine.py `District`	Synthetic: elevation, river distance, drainage, population, vulnerability, Voronoi village assignment. Replace with real rasters.
Flood model	engine.py `FloodModel`	Logistic regression on 7 features (elevation, river distance, drainage, rainfall + 3 rain interactions), trained on 7,000 simulated events; reports held-out accuracy and ROC-AUC; feature weights are shown to users (explainable).

Component	File	Notes
Routing	engine.py `graph` , `_nav`	Sparse risk-weighted graph; SciPy `dijkstra(min_only=True)` .
Optimizer	engine.py `optimize`	Greedy maximum coverage over flood-safe candidate sites, weight = people x P x (0.6 + vulnerability), 30-minute walk radius, soft capacity cap.
Early warning	engine.py `forecast`	72 h hourly rain → rolling 24 h rain → model → village risk per hour → first hour above 35% = danger hour → timed action plan.
Alerts	engine.py `alert_messages` , app.py	English / Hindi / Tamil templates; send is simulated and logged (gateway plug-in point).
Supplies	engine.py `supplies`	3 days: 3 L water and 1 food packet per person per day, 1 medical kit / 100 people, 1 boat / 150 stranded, 1 tarpaulin / 5 people.
API + DB	app.py	Flask REST, SQLite tables `reports` , `alerts` , `scenarios` . Input validation on all endpoints.
Frontend	static/	Vanilla JS; canvas map with bilinear terrain shading; EN/Hindi toggle; guided tour.

5. API summary

`GET /api/district` · `GET /api/model` · `POST /api/analyze` · `POST /api/optimize` · `GET /api/forecast?scenario=cyclone|monsoon|cloudburst|(live=1&lat=&lon=)` `GET|POST /api/reports` · `POST /api/reports/<id>/vote|resolve` · `POST /api/alerts/preview|send` · `GET /api/alerts/log` `GET|POST /api/scenarios` · `GET /api/export/supplies.csv` · `GET /api/health`
Example requests and real outputs: `samples/` .

6. Testing

`python test_api.py` runs 14 checks: district layers, model AUC, analyze KPIs, more rain → more people affected, optimizer returns k sites, optimized coverage beats the naive layout, forecast, report create/validate/list, alert preview and send, scenario save, CSV export.

7. Limitations and honest notes

- All district data and "historical floods" are synthetic; the model is not validated on real floods.
- Walking-time routing on a 250 m grid is an approximation (no road network, no vehicles).
- Alert sending is simulated. Live forecast needs internet and was not tested in the build sandbox.

8. Path to production

Replace `District` with SRTM / CartoDEM, IMD / CHIRPS rainfall, WorldPop / Census population, Sentinel-1 flood extents and OpenStreetMap roads; retrain on observed flood labels (gradient boosting or U-Net); connect SMS / IVR / WhatsApp; add authority roles, audit log and privacy-preserving household data.

Team Eunoia · HSC|TN|02046 · FloodShield